Practical No.6

Aim: Write a JSP program for 1.Tag (JSTL ,Custom Tags) 2. Exception handling.

Theory:

The **JSP Standard Tag Library (JSTL)** is a collection of ready-made tags that simplify JSP development by replacing Java code (scriptlets) with clean, readable tags.

JSTL (JavaServer Pages Standard Tag Library) provides standard tags for:

- Logic (if, loop)
- Database operations
- String formatting
- XML processing

1. Core Tags (`c`): Used for basic operations like conditions and loops.

Tag	Description
<c:out>	Display data
<c:set>	Set variable
<c:if>	Conditional
<c:forEach>	Loop
<c:choose>	Switch-case

2. **Formatting Tags (`fmt`):**Used for formatting text, numbers, dates.

Tag	Description
<fmt:formatNumber>	Format numbers
<fmt:formatDate>	Format date
<fmt:setLocale>	Set language

3. **SQL Tags (`sql`):**Used for database operations (not recommended in real apps, only learning).

Tag	Description
<sql:setDataSource>	Connect DB
<sql:query>	Execute query
<sql:update>	Insert/update

Custom Tags: JavaServer Pages (JSP) provides custom tags to simplify JSP pages by moving Java code out of JSP files and into reusable Java classes. Custom tags help developers create clean, readable, and maintainable JSP code by replacing complex scriptlets with meaningful tag names.

Custom tags are defined using:

- Tag Handler Class
- Tag Library Descriptor (TLD) file
- JSP page that uses the custom tag

TagSupport Class: The TagSupport class is a part of the **javax.servlet.jsp.tagext** package. It is a convenient base class for developing custom tag handlers.

Key features of TagSupport:

- Provides default implementations of tag lifecycle methods
- Simplifies custom tag creation
- Supports attributes and body content
- Allows reuse of tags across JSP pages

Important lifecycle methods:

- doStartTag() – executed when the tag is encountered
- doEndTag() – executed after the tag body is processed

Tag Library Descriptor (TLD):A Tag Library Descriptor (TLD) is an XML file that describes custom tags to the JSP container.

The TLD file contains:

- Tag name
- Tag handler class
- Attributes of the tag
- Tag library URI

The TLD acts as a bridge between JSP pages and tag handler classes.

Exception Handling in JSP: Exception handling in JSP allows developers to handle runtime errors gracefully without breaking the application.

Common ways to handle exceptions:

- Using try–catch blocks in tag handler classes
- Using JSP error pages
- Throwing JspException or IOException

In custom tags, exception handling is typically done inside the doStartTag() or doEndTag() methods to handle invalid inputs or runtime issues.

Custom Tag with Exception Handling

When custom tags process user input or perform calculations, exceptions may occur (e.g.,

invalid data, division by zero). Handling exceptions inside the tag handler ensures:

- Application stability
- Meaningful error messages
- Clean JSP code without Java scriptlets

Conclusion:

Problem Statement – 1 : Design and develop a JSP application to demonstrate Custom Tag creation using TagSupport. The custom tag should perform a simple operation such as displaying a message and numerical operation for displaying square of a number.

customtag.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
  pageEncoding="UTF-8"%>
  <%@ taglib   uri="/WEB-INF/customtag.tld"   prefix="ctag" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
<h1>Custom Tag Example</h1>
<h2>1.HelloTag</h2>
<ctag:hellotag />
<h2>2.SquareTag</h2>
<ctag:square number="4" />
<h2>2.SquareTag</h2>
<ctag:square number="5" />
</body>
</html>
```

customtag.tld

```
<?xml version="1.0" encoding="UTF-8"?>
<taglib>
        <jsp-version>3.0</jsp-version>
        <tlib-version>1.0</tlib-version>
        <tag>
        <name>hellotag</name>
```

```
        <tag-class>p1.HelloTag</tag-class>
        <body-content>empty</body-content>

  </tag>

  <tag>
      <name>square</name>
      <tag-class>p1.SquareTag</tag-class>
      <body-content>empty</body-content>
      <attribute>
      <name>number</name>
      <required>true</required>
      </attribute>
  </tag>
</taglib>
```

HelloTag.java

```
package p1;
//import jakarta.servlet.jsp.tagext.TagSupport;
//import jakarta.servlet.jsp.JspException;
//import jakarta.servlet.jsp.JspWriter;

import javax.servlet.jsp.tagext.TagSupport;
import javax.servlet.jsp.JspException;
import javax.servlet.jsp.JspWriter;
import java.io.IOException;

public class HelloTag extends TagSupport {
   @Override
  public int doStartTag() throws JspException {
     try {
        JspWriter out = pageContext.getOut();
        out.println("<h3>" + "This message is from HelloTag" + "</h3>");
     } catch (IOException e) {
        throw new JspException(e);
```

```
    }
    return SKIP_BODY;
  }
}
```

SquareTag.java

```
package p1;
import javax.servlet.jsp.tagext.TagSupport;
import javax.servlet.jsp.JspException;
import javax.servlet.jsp.JspWriter;
import java.io.IOException;

public class SquareTag extends TagSupport {
        private int number;
        public void setNumber(int number) {
                this.number=number;
        }
        @Override
        public int doStartTag()throws JspException
        {
                try {
                        JspWriter out=pageContext.getOut();
                        int square=number*number;
                        out.println("Square of a number is" + square);

                }catch(IOException e) {
                        throw new JspException(e);
                }
                return SKIP_BODY;
        }
```

Problem Statement – 2 Demonstration of Exception handling in JSP Using "errorpage & iserrorpage" attribute of @page directive.

exception.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
  pageEncoding="ISO-8859-1" errorPage="error1.jsp"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<%
int a=10,b=0,c=0;
c=a/b;
%>
</body>
</html>
```

error1.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
```

```
  pageEncoding="ISO-8859-1" isErrorPage="true"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<%= exception %>
</body>
</html>
```

Practical No.7

Aim: Design and develop JSP application using JSP Elements(directives,action elements,custom tags,implicite object) and JDBC.

Problem Statement:

A1A2: Student Registration System:

- Design and develop a JSP web application to register students by entering details: such as Student ID, Name, Course, and Marks.
- Use JSP directives for page configuration, action elements to manage JavaBeans, implicit objects to handle request and response, and JDBC to store and retrieve student details from a database.
- Implement a custom tag to calculate and display the student grade.

A3A4: Employee Information Management System:

- Develop a JSP application that allows users to add and view employee information such as Employee ID, Name, Department, and Salary.
- Use directives for importing packages, action tags for bean manipulation, implicit objects for handling form data, and JDBC to store employee data in a database.
- Create a custom tag to calculate employee bonus.

B1B2: Product Management System

- Design a JSP application to manage product information including Product ID, Product Name, Price, and Quantity.
- Use JSP directives, action elements, and implicit objects to process user input. Store product data in a database using JDBC .
- create a custom tag to calculate the total price of products.

B3B4: online bookstore application

- Develop a JSP-based online bookstore application where users can view and purchase books.
- Use JSP directives for including common pages, action tags for navigation between pages, implicit objects for session management, and JDBC to retrieve book details from the database.
- Implement a custom tag to calculate the total bill.

C1C2: Feedback Management System

- Create a JSP application that collects user feedback including Name, Email, and Comments.
- Use JSP directives and action tags for page processing, implicit objects to handle form data, custom tags to format the feedback output, and JDBC to store feedback information in a database.

C3C4: Course Registration System

- Develop a JSP application for course registration where students can select courses and submit their details.
- Use JSP directives, action tags, and implicit objects to process form data. Use JDBC to store registration details in a database and create a custom tag to calculate total course fee.